联想·吠云·泰坦 运动控制SDK

1、SDK更新记录

版本号	修改时间	修改内容
V1.0,0	20251017	首次发布
V1.0.1	20251101	1、修改了SDK使用设备的IP地址 2、增加了轮速数据的订阅。
V1.0.2	20251110	1、系统状态中将原错误码分离错误码和警告码 2、系统状态中的 version 和 serialNumber 移除，新增 AstrallGetDeviceInfo 获取设备信息接口 3、AstrallSdkInit 传参中添加 SDK 状态回调接口 4、接口修改为阻塞式，直至响应或超时 5、修改控制权机制，详见关于控制权章节 6、未连接时调用 AstrallGetSportStatus 将返回 ASTRALL_SPORT_STATUS_UNKNOWN 状态
V1.0.3	20251113	1、增加了订阅泰坦头部相机RGB数据的接口
V1.0.4	20251114	1、增加向遥控器消息窗口推送消息的接口 AstrallSendMessage 2、增加订阅摇杆数据
v1.0.5	20251125	1、修改了控制权的获取逻辑 2、增加了自动充电任务的相关接口和状态 3、增加了站立中和趴下中两个状态 4、增加了当速度指令不下发时，自动停止，现在同时支持发0停止和自动停止 5、在imu的数据结构中加入了里程计数据x，y 6、增加了6.10.1节机动模式切换的相关描述

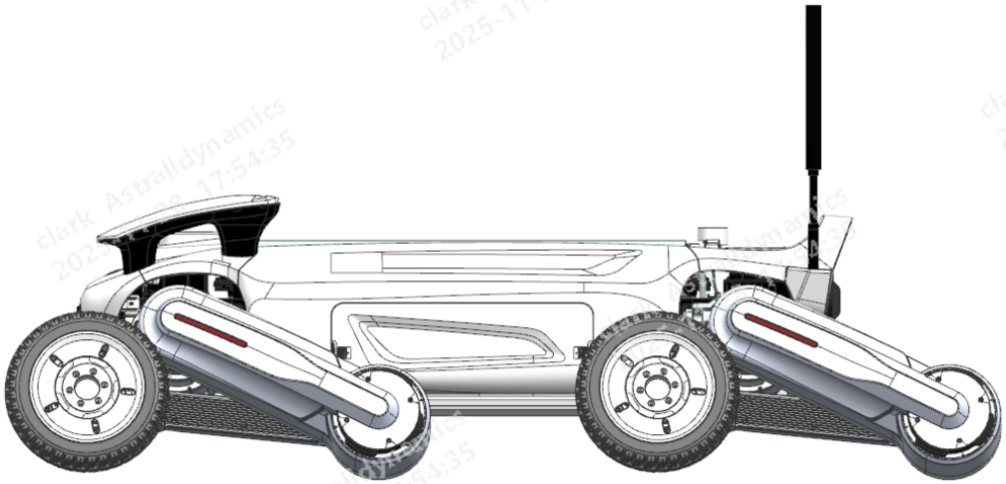
2、SDK概述

泰坦SDK提供了与泰坦运动控制模块的交互接口，开发者可以通过泰坦SDK控制机器人完成运动、姿态切换以及获取泰坦的状态信息，SDK通过UDP 与运控模块进行通讯，支持多平台使用。

SDK主要功能包括：

- 运动控制：前进、后退、左转、右转、左平移、右平移；
- 姿态控制：锁定关节站立、锁定关节趴下、软急停、全地形行走、避障开关等；
- 状态获取：各关节位置、速度、温度、电池信息、运动模式、状态码等
- 图传接口：获取一帧RGB图像或一帧深度图像、一帧雷达点云图像

clark Astralldynamics 2025-11-28 17:54:35



锁定关节趴下

锁定关节站立

> *泰坦趴下姿态整体尺寸为0.98m*0.61m*0.48m（长宽高）*

在使用SDK与泰坦进行通讯时，请确保泰坦机器人处于阻尼模式或锁定关节趴下状态。

当SDK与泰坦运动控制模块建立连接后，运动控制模块会优先执行SDK下发的控制指令，当SDK没有指令下发超过一定时间后，运动控制模块会拿回本体控制权，进入阻尼模式一段时间后，清空SDK指令。

3、调试方式

clark Astralldynamics 2025-11-28 17:54:35

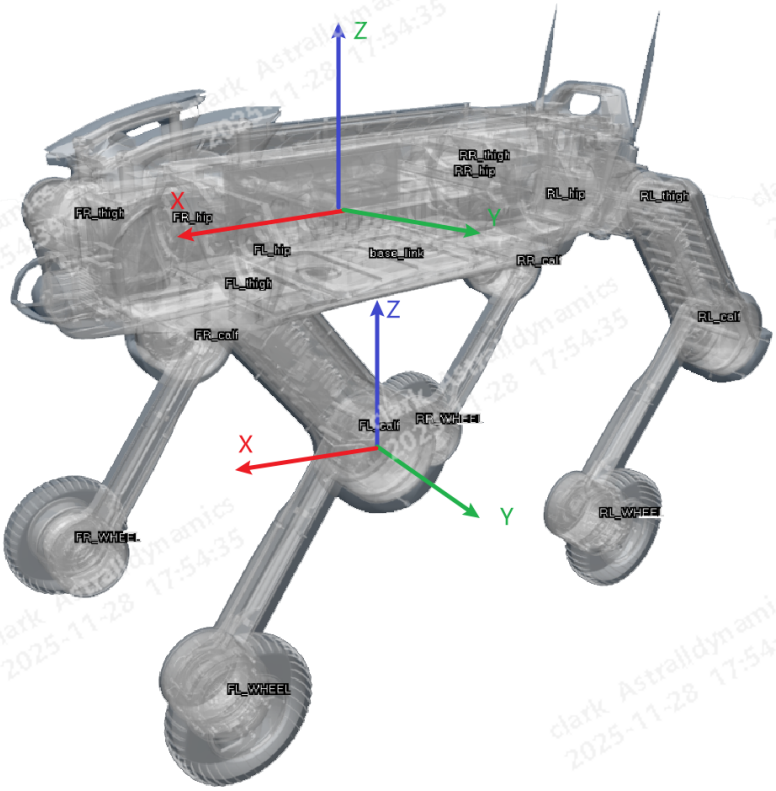
- 通讯协议：UDP
- 泰坦默认IP：10.18.0.100
- 调用SDK设备IP：10.18.0.200
- 泰坦接收端口：3600
- 调用SDK设备接收数据端口：3600
- 数据格式：结构化数据包（小端序）
- 头部雷达IP： 10.18.0.120 MSOP端口：6699 DIFOP端口：7788 IMU获取端口：6688
- 尾部雷达IP： 10.18.0.121 MSOP端口：6969 DIFOP端口：7878 IMU获取端口：6868

4、硬件参数

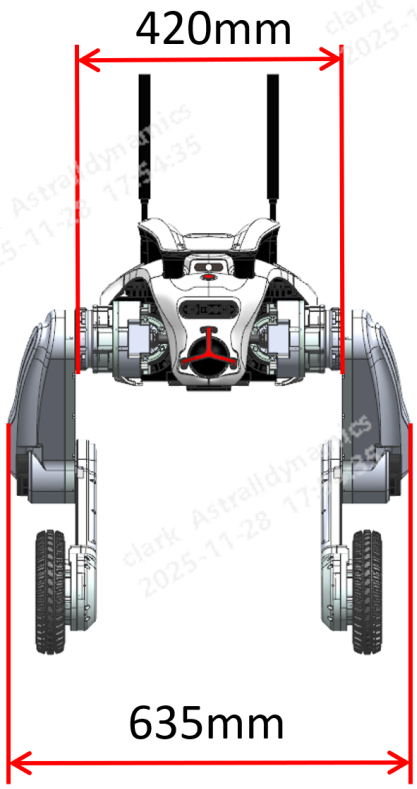
4.1、本体坐标系与关节坐标系

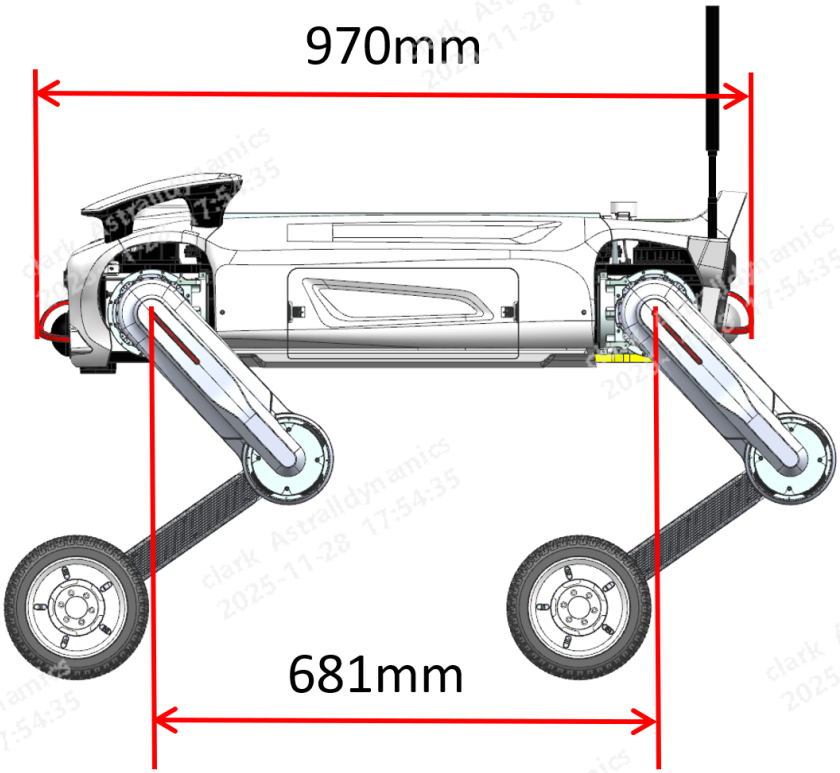
机身坐标系（本体系）定义：X轴朝向头部，Y轴朝向机身左侧，Z轴沿XxY朝向机器狗上方。

各个关节坐标系定义：**各个关节的坐标系与本体系定义一致**，其中，机身关机的旋转轴为 x 轴，大腿关节(髋关节)和小腿关节(膝关节)的旋转轴为 y 轴，旋转正方向符合右手定则。



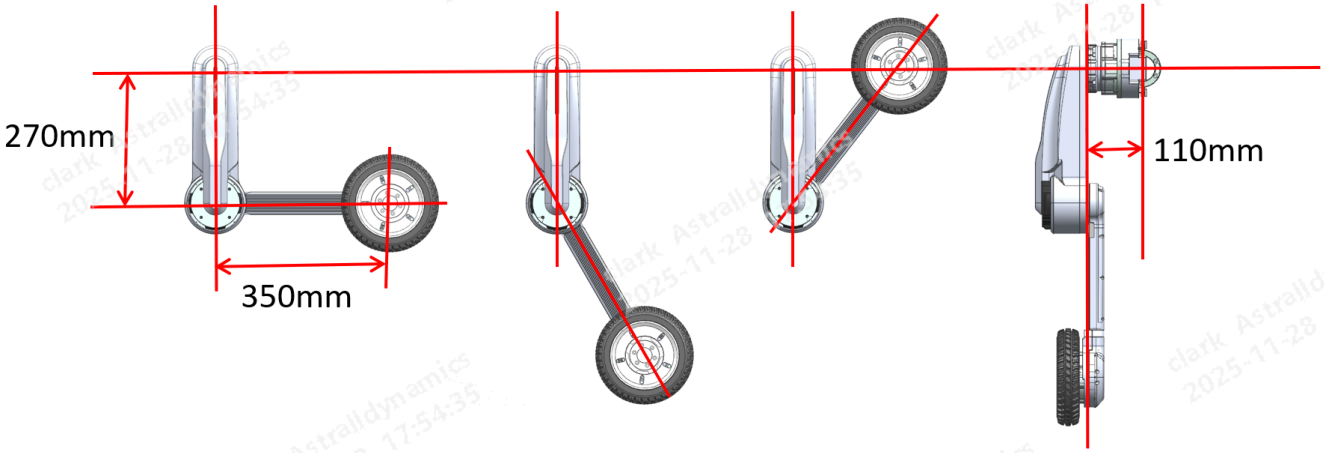
4.2、身体连杆参数





参数	数值	说明
长度	0.97m	身体总长度
腿平面左右间距	0.681m	腿平面左右的距离
宽度	0.635m	身体总宽度

4.3、腿部连杆参数



参数	数值	说明
腿平面与髋侧摆关节距离(L1)	0.11m	髋侧摆关节与腿平面距离
大腿长度(L2)	0.27m	髋前摆关节中心与膝关节中心距离
小腿长度(L3)	0.35m	膝关节中心与足底圆心距离
轮半径	0.0925m	轮半径

4.4、关节参数

关节	运动范围	额定扭矩	额定转速	峰值扭矩
髋关节(Hip)	-15° ~15°	50N·m	100rpm±10%	300N·m
大腿关节(Thigh)	-170° ~170°			
小腿关节(Calf)	-148° ~148°			

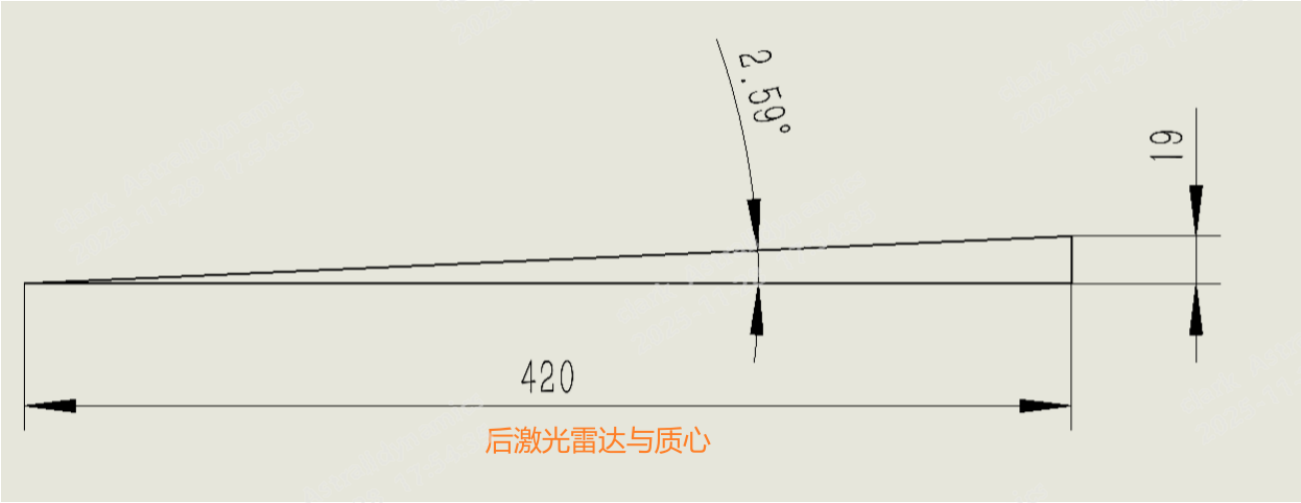
> *注意：泰坦机器人的其他动力学参数可以在提供的URDF文件中获取。*

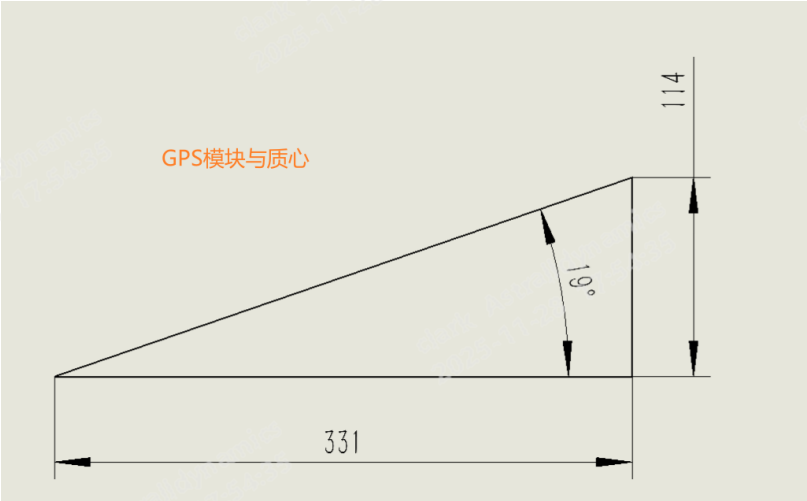
4.5、传感器外参

传感器坐标相对于质心坐标在机体系(4.1节中定义)下的位置，质心（0，0，0）

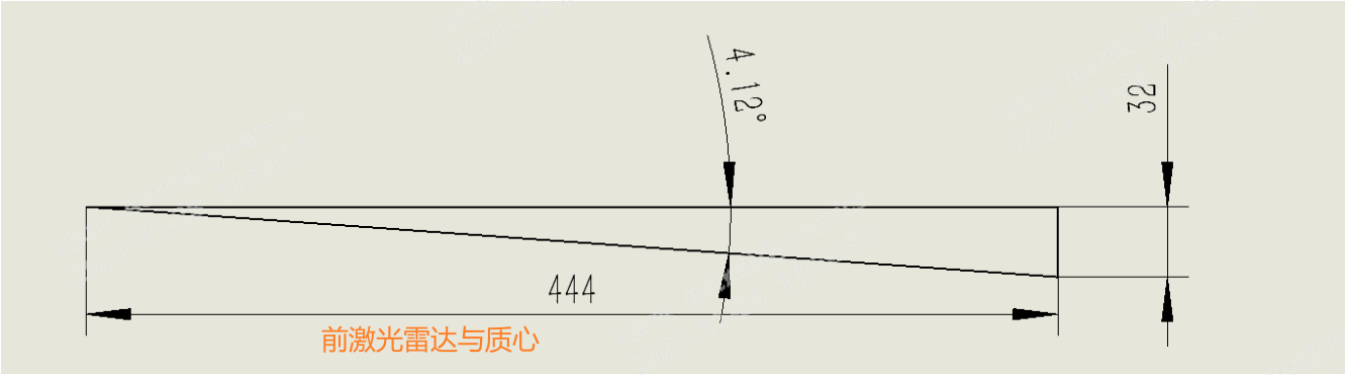
传感器	坐标
前激光雷达	444，0，-32
后激光雷达	-420，0，19
双目相机	447，0，41
IMU	127，-85，82
GPS	331，0，114

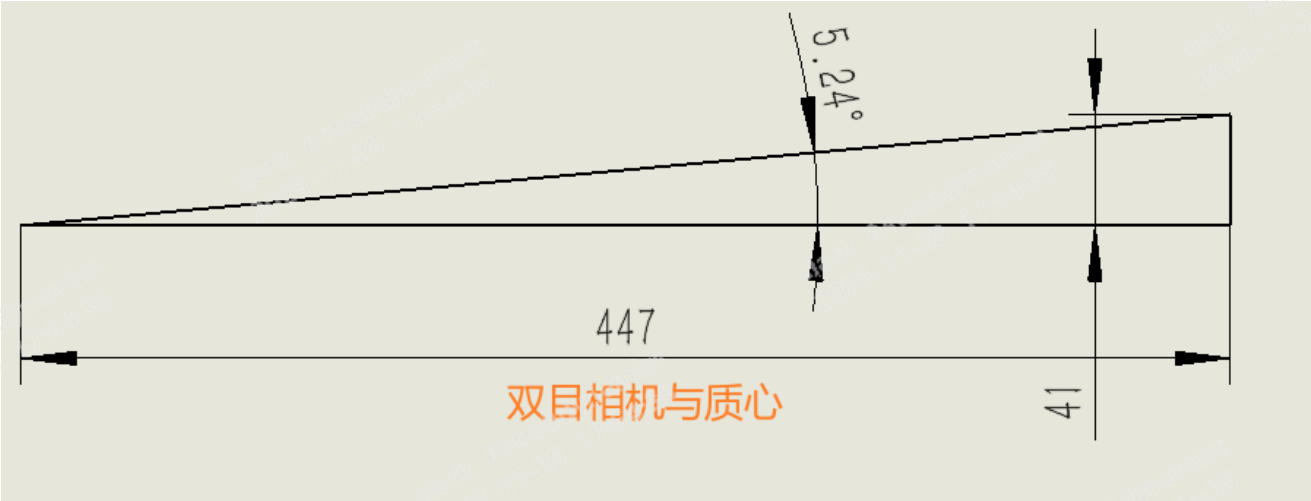
角度关系





clark Astralldynamics 2025-11-28 17:54:35





5、SDK获取

```
Git
// Contact the technical staff for assistance.
```

6、软件服务接口

泰坦SDK提供订阅/推送、请求/应答两种模式。

订阅/推送：接收方通过泰坦SDK订阅某个消息，机器狗通过订阅内容向接收方推送消息，用于中高频数据(如IMU)或持续的数据交互；

请求/应答：问答模式，通过请求实现模型切换、姿态控制等操作，用于低频或功能切换时的数据交互。

泰坦SDK对大部分功能进行了优化封装，实现函数式调用。

函数调用返回参数表

变量原型	值	含义
ASTRALL_RES_FAILED	0x8004	执行失败/申请失败/订阅失败
ASTRALL_RES_TIMEOUT	0x8005	超时
ASTRALL_RES_RUNNING	0x8006	执行中
ASTRALL_RES_SUCCESSED	0x8008	执行成功

ASTRALL_RES_INVALID_PARAM	0x8010	无效参数
ASTRALL_RES_RC_NORELEASE	0x8020	遥控器抢夺未释放控制权
ASTRALL_RES_BEEN_OBTAINED	0x8021	已经有设备获得控制权未释放
ASTRALL_RES_WITHOUT_AUTH	0x8022	没有控制权

6.1 SDK初始化

函数名	AstrallSdkInit
函数原型	uint16_t AstrallSdkInit(AstrallConfig& cfg, uint32_t timeout)
功能描述	阻塞式调用，直到建立连接或超时
使用方法	请求/应答

参数	struct AstrallConfig
	{
	// 心跳回调函数，收到泰坦返回的心跳ACK后执行此函数。
	// data：无
	std::function<void(void *data, uint16_t len)> heartbeatCb;
	// SDK状态回调函数
	// data：见下 AstrallSdkStatus 结构体数据
	std::function<void(void *data, uint16_t len)> sdkStatusCb;
	};
	timeout: 超时时间（单位：ms）

返回值	详见 函数调用返回参数表
备注	无

SDK 状态

```
struct AstrallSdkStatus {
    uint8_t link : 1;    // Connection status
    uint8_t ctrlAuthority : 1;  // ctrl authority status
    uint8_t reserve : 6;

    AstrallSdkStatus()
    {
        link = 0;
        ctrlAuthority = 0;
        reserve  = 0;
    }
};
```

6.2 SDK反初始化

函数名	AstrallSdkDeinit
函数原型	void AstrallSdkDeinit()
功能描述	断开连接，清理释放资源
参数	无
返回值	详见 函数调用返回参数表
备注	无

6.3 心跳接口

函数名	AstrallHeartbeat
函数原型	uint16_t AstrallHeartbeat(uint32_t timeout)
功能描述	发送心跳包
使用方法	请求/应答
参数	timeout: 超时时间（单位：ms）
返回值	详见 函数调用返回参数表
备注	心跳接收在初始化时的回调中，推荐心跳频率为10Hz， 若500ms内泰坦未收到心跳包，则清0SDK下发的指令， 保持站立姿态；若再接下来的500ms内仍未收到心跳包， 泰坦判断SDK断开连接，清除SDK连接标识，清除SDK控制权， 保持站立姿态，等待其他控制器指令， 若无其他控制器(如遥控器未连接)则控制泰坦机器人趴下。

6.4 设备信息接口

函数名	AstrallGetDeviceInfo
函数原型	uint16_t AstrallGetDeviceInfo(AstrallDeviceInfo& info, uint32_t timeout)
功能描述	获取设备信息
使用方法	请求/应答
参数	info：内容及结构见下方 timeout：超时时间（单位：ms）
返回值	详见 函数调用返回参数表
备注	无

设备信息

```
struct AstrallDeviceInfo
{
    std::string version;
    std::string serialNumber;
    uint32_t model;

    AstrallDeviceInfo()
    {
        version = "";
        serialNumber = "";
        model = 0;
    }
};
```

6.5 系统信息接口

函数名	AstrallGetSystemStatus
函数原型	uint16_t AstrallGetSystemStatus(AstrallSystemStatus& status)
功能描述	获取系统状态参数

使用方法	请求/应答
参数	status：内容及结构见下方
返回值	详见 函数调用返回参数表
备注	SDK 状态中 link 为连接状态时，返回数据才有效

SystemStatus结构

```
enum AstrallSystemCode : uint8_t
{
    ASTRALL_SYSTEM_CODE_INIT     = 0,   // Initialization & System Self-Check
    ASTRALL_SYSTEM_CODE_STANDBY  = 1,   // Standby -> Damping Mode
    ASTRALL_SYSTEM_CODE_RUNNING  = 2,   // Normal Operation
    ASTRALL_SYSTEM_CODE_WARNING  = 3,   // Warning -> Need to be handled as soon as possible
    ASTRALL_SYSTEM_CODE_ERROR    = 4,   // Error -> Emergency Shutdown
    ASTRALL_SYSTEM_CODE_SHUTDOWN = 5    // Shut Down
};

enum AstrallErrorCode : uint32_t
{
    ASTRALL_ERR_NONE                = 0,
    ASTRALL_ERR_MOTOR_VERSION       = (1U << 0),    // Motor driver version error
    ASTRALL_ERR_MOTOR_COMM_LOSE     = (1U << 1),    // Motor communication lost
    ASTRALL_ERR_MOTOR_STATUS        = (1U << 2),    // Motor abnormal. Please check the motor status
    ASTRALL_ERR_MOTOR_OVER_TEMP     = (1U << 3),    // Motor over temperature
    ASTRALL_ERR_STO_TRIGGERED       = (1U << 4),    // STO triggered
    ASTRALL_ERR_IMU_ERROR           = (1U << 5),    // IMU error
    ASTRALL_ERR_LOW_BATTERY_LEVEL   = (1U << 6),    // Battery level too low
    ASTRALL_ERR_OVER_LOAD           = (1U << 7),    // Motor overload
    ASTRALL_ERR_MBUS_DRIVER_FAIL    = (1u << 8),    // Driver version error
};

enum AstrallWarnCode : uint32_t
{
    ASTRALL_WARN_NONE               = 0,
    ASTRALL_WARN_MOTOR_OVER_TEMP    = (1U << 0),    // Motor about to overheat
    ASTRALL_WARN_JOYSTICK_LOCK      = (1U << 1),    // Remote controller is locked. Switch to the third gear to
unlock
    ASTRALL_INFO_JOYSTICK_INTERVENTION = (1U << 2), // Remote controller has obtained control
    ASTRALL_WARN_MOTOR_COMM_UNSTABLE = (1U << 3),   // Motor communication unstable
    ASTRALL_WARN_LOW_BATTERY_LEVEL  = (1U << 4),    // Low battery level warning
    ASTRALL_WARN_JOYSTICK_LOSE      = (1U << 5),    // Remote controller offline
    ASTRALL_WARN_MOTOR_OVER_LOAD    = (1U << 6),    // Motor about to overload
    ASTRALL_WARNING_CONNECT_SDK_LOSE  = (1u << 7),  // Connected SDK is offline
    ASTRALL_WARNING_LOSE_CAMERA       = (1u << 8),  // No USB camera device was obtained.
};

struct AstrallSystemStatus
{
    AstrallSystemCode sysStatus;        // System status word
    AstrallErrorCode errorCode;         // Error Code
    AstrallWarnCode warnCode;           // Warn Code

    AstrallSystemStatus()
    {
        sysStatus = AstrallSystemCode::ASTRALL_SYSTEM_CODE_INIT;
        errorCode = AstrallErrorCode::ASTRALL_ERR_NONE;
        warnCode = AstrallWarnCode::ASTRALL_WARN_NONE;
    }
};
```

错误码说明：当有任何一个错误码出现，机器狗整机立即趴下，不再响应任何控制指令

警告码说明：

警告码	含义	说明
ASTRALL_WARN_MOTOR_OVER_TEMP	电机即将过温	电机温度即将超过85℃或驱动温度超过70℃，继续大负载动作可能会触发电机高温保护（电机120摄氏度）
ASTRALL_WARN_JOYSTICK_LOCK	遥控器已锁定，开关拨至三档可解锁	不影响使用
ASTRALL_INFO_JOYSTICK_INTERVENTION	遥控器已获取控制权	如需要再次获取控制权，可以重新申请
ASTRALL_WARN_MOTOR_COMM_UNSTABLE	电机通讯不稳定	不影响使用
ASTRALL_WARN_LOW_BATTERY_LEVEL	电池电量不足警告	提示用户电量即将不足，不影响使用
ASTRALL_WARN_JOYSTICK_LOSE	遥控器离线	不影响使用
ASTRALL_WARN_MOTOR_OVER_LOAD	电机即将过载	提示用户电机即将过载，继续进行大负重运动、上楼梯、爬坡等动作可能会造成电机过载。

6.6 电池信息接口

函数名	AstrallGetPowerStatus
函数原型	uint16_t AstrallGetPowerStatus(AstrallPowerStatus& power)
功能描述	获取电源参数
使用方法	请求/应答
参数	power：内容及结构见下方
返回值	详见 函数调用返回参数表
备注	SDK 状态中 link 为连接状态时，返回数据才有效

电池信息

```
struct AstrallPowerStatus
{
    float soc;                  // Remaining battery power: %
    float temp;                 // Battery temperature
    float voltage;              // Battery voltage
    uint16_t cycleCount;        // Number of charge cycles
    uint16_t charged;           // Charging status 0:unknown 1:discharged 2:stand 3:fast

    AstrallPowerStatus()
    {
        soc = 0.0f;
        temp = 0.0f;
        voltage = 0.0f;
        cycleCount = 0;
        charged = 0;
    }
};
```

6.7 当前模式信息

函数名	AstrallGetSportStatus
函数原型	uint16_t AstrallGetSportStatus()
功能描述	获取泰坦当前运动状态

使用方法	请求/应答
参数	无

返回值	enum AstrallSportStatus : uint16_t
	{
	
	ASTRALL_SPORT_STATUS_UNKNOWN = 0, // 未知状态
	ASTRALL_SPORT_STATUS_INIT = 0xB100, // 初始化中
	ASTRALL_SPORT_STATUS_DAMPING = 0xB101, // 阻尼状态
	ASTRALL_SPORT_STATUS_FIXEDSTAND = 0xB102, // 锁定站立状态
	ASTRALL_SPORT_STATUS_FIXEDDOWN = 0xB103, // 锁定趴下状态
	ASTRALL_SPORT_STATUS_MOVE = 0xB104, // 行走(RL步态全地形覆盖)中
	
	ASTRALL_SPORT_STATUS_UPSTAIRS = 0xB105, // 爬楼梯中
	ASTRALL_SPORT_STATUS_CAR = 0xB106, // 负重小车模式
	ASTRALL_SPORT_STATUS_AUTOCHAEGE = 0xB107, // 准备执行自动充电
	ASTRALL_SPORT_STATUS_SEARCHCHARGING = 0xB108, // 寻找充电桩中
	ASTRALL_SPORT_STATUS_PLUGGINGCHARGING = 0xB109, // 已找到，正在上桩
	ASTRALL_SPORT_STATUS_ATCHARGING = 0xB10A, // 在充电桩上
	ASTRALL_SPORT_STATUS_EXITCHARGING = 0xB10B, // 退出充电桩中
	
	ASTRALL_SPORT_STATUS_CHARGING = 0xB10C，// 充电中
	
	ASTRALL_SPORT_STATUS_FIXEDSTANDING = 0xB10D, // 执行站立中
	ASTRALL_SPORT_STATUS_FIXEDDOWNING = 0xB10E, // 执行趴下中
	ASTRALL_SPORT_STATUS_GET_RIGHT = 0xB1FF // 摔倒自主恢复站立
	};

备注	SDK 状态中 link 为连接状态时，返回数据才有效

6.8 传感器信息接口

传感器数据获取方式为 订阅/推送

函数名	**AstrallSubscriptionData**
函数原型	uint16_t AstrallSubscriptionData(AstrallSubscribeTopicId topicId, AstrallSubscribeFreq freq, std::function<void(void *data, uint16_t len)> cb, uint32_t timeout)
功能描述	订阅传感器数据
使用方法	订阅/推送
参数	topicId：订阅传感器ID freq：需要获取的传感器数据的频率 cb：注册传感器数据回调函数，当接收到传感器数据返回时调用此函数 timeout：超时时间（单位： ms）
返回值	详见 函数调用返回参数表
备注	无

话题列表

```
enum AstrallSubscribeTopicId : uint16_t
{
    ASTRALL_SUB_TOPIC_ID_IMU        = 0x0001U,    // Subscribe IMU Data
    ASTRALL_SUB_TOPIC_ID_SPORT      = 0x0002U,    // Subscribe Sport Data -> wheel speed
    ASTRALL_SUB_TOPIC_ID_RGB        = 0x0003U,    // Subscribe Camera RGB to point:6000
    ASTRALL_SUB_TOPIC_ID_JOYSTICK   = 0x0010U,    // Subscribe joystick data
};


enum AstrallSubscribeFreq : uint16_t
{
    ASTRALL_SUB_FREQ_CLOSE          = 0U,
    ASTRALL_SUB_FREQ_1HZ,
    ASTRALL_SUB_FREQ_25HZ,
    ASTRALL_SUB_FREQ_50HZ,
    ASTRALL_SUB_FREQ_125HZ,
    ASTRALL_SUB_FREQ_250HZ
};
```

TOPIC_ID	支持频率（Hz）	回调 data 数据结构	说明
ASTRALL_SUB_TOPIC_ID_IMU	1、25、50、125、250	AstrallImuData	--
ASTRALL_SUB_TOPIC_ID_SPORT	1、25、50、125、250	AstrallSportData	--
ASTRALL_SUB_TOPIC_ID_RGB	--	--	订阅频率填除0外的任意字段，（填0为取消订阅），订阅成功后会通过UDP 向 SDK IP (10.18.0.200)的端口6000推送相机 H264 格式 RGB码流。
ASTRALL_SUB_TOPIC_ID_JOYSTICK	50	AstrallJoystickData	--

IMU数据获取接口

函数名	AstrallGetImuData
函数原型	uint16_t AstrallGetImuData(AstrallImuData& data)
功能描述	获取IMU数据
使用方法	订阅IMU数据后调用此函数，未订阅则调用后无法获取数据
参数	data：内容及结构见下方
返回值	详见 函数调用返回参数表
备注	订阅 IMU 数据后该函数返回数据才有效

回调Data数据结构

```
struct AstrallImuData
{
    long long timestamp;        // IMU timestamp
    float accelerometer[3];     // Acceleration: m/(s*s)
    float gyroscope[3];         // Angular velocity: rad/s
    float quaternion[4];        // Quaternion
    float pitch;                // Pitch angle: rad
    float roll;                 // Roll angle: rad
    float yaw;                  // Heading angle: rad
    float odomX;
    float odomY;

    AstrallImuData()
    {
        timestamp = 0;
        memset(accelerometer, 0, sizeof(accelerometer));
        memset(gyroscope, 0, sizeof(gyroscope));
        memset(quaternion, 0, sizeof(quaternion));
        pitch = 0.0f;
        yaw = 0.0f;
        roll = 0.0f;
        odomX = 0.0f;
        odomY = 0.0f;
    }
};

struct AstrallSportData
{
    long long timestamp;
    float wheelSpeed[4];

    AstrallSportData()
    {
        timestamp = 0;
        memset(wheelSpeed, 0, sizeof(wheelSpeed));
    }
};

struct AstrallJoystickData
{
    long long timestamp;
    int16_t rocker[6];
    uint16_t l1 : 1;
    uint16_t l2 : 1;
    uint16_t l3 : 1;
    uint16_t r1 : 1;
    uint16_t r2 : 1;
    uint16_t r3 : 1;
    uint16_t sl : 1;
    uint16_t sr : 1;
    uint16_t h : 1;
    uint16_t sw : 2;
    uint16_t reserve : 5;
};
```

雷达数据需要 调用厂家SDK或通过UDP在对应端口获取点云、雷达内置IMU数据。

6.9 向遥控器消息窗口发送消息

函数名	AstrallSendMessage
函数原型	uint16_t AstrallSendMessage(char *data, uint16_t len)
功能描述	向遥控器消息窗口发送消息

使用方法	--
参数	data：文本消息 len：消息长度
返回值	详见 函数调用返回参数表
备注	该函数调用频率应小于 5 Hz，否则返回失败

6.10 控制模块

6.10.1 机动模式切换接口

函数名	AstrallSportModeControl
函数原型	uint16_t AstrallSportModeControl(AstrallSportModeCmd mode, uint32_t timeout)
功能描述	控制泰坦切换不同的模式
使用方法	请求/应答
参数	mode：结构及内容见下方 timeout：超时时间（单位： ms）
返回值	详见 函数调用返回参数表
备注	1、全地形行走模式(Move)，只能在机器狗完成站立后切换，否则返回失败； 2、自动充电指令只能在**站立模式**或**全地形行走模式**中才能执行，否则返回失败； 3、退桩指令，只能在机器狗状态处于充电中或在充电桩上才能执行，否则返回失败； 4、机器狗处于**在桩上**或**充电中**不能执行趴下动作； 5、机器狗在任意模式中都可以执行急停，进入阻尼模式；

运动模式

```
enum AstrallSportModeCmd : uint16_t
{
    ASTRALL_SPORT_MODE_CMD_DAMPING       = 0xA101,       // A101H Damping
    ASTRALL_SPORT_MODE_CMD_FIXEDSTAND    = 0xA102,       // A102H Fixed Stand
    ASTRALL_SPORT_MODE_CMD_FIXEDDOWN     = 0xA103,       // A103H Fixed Down
    ASTRALL_SPORT_MODE_CMD_MOVE          = 0xA104,       // A104H Move
    ASTRALL_SPORT_MODE_CMD_AUTOCHARGE    = 0xA105,       // A105 Auto Charge
    ASTRALL_SPORT_MODE_CMD_EXITCHARGE    = 0xA106,       // A106 Exit Charge
    ASTRALL_SPORT_MODE_CMD_GET_RIGHT     = 0xA1FF        // A1FFH Recover the ability to stand independently
after falling down
};
```

6.10.2 控制权申请接口

函数名	AstrallAuthControl
函数原型	uint16_t AstrallAuthControl(AstrallAuth auth, uint32_t timeout)
功能描述	申请泰坦控制权
使用方法	请求/应答
参数	auth：控制权 ID timeout：超时时间（单位：ms）
返回值	详见 函数调用返回参数表
备注	无

控制权说明

控制权：是指当前具备控制机器狗的权限，控制对象有PSDK、遥控器等，控制权允许申请、抢夺和主动释放。

1、在调用机器狗控制接口前，需要先调用**AstrallAuthControl()**接口申请控制权，在获得机器狗控制权之前，机器狗的控制接口将无法生效。

2、失控行为：当机器狗在控制过程中由于各种突发因素导致的不受控状态（如遥控器断联、SDK程序崩溃、机器狗低电量等场景）。进入失控模式后，机器狗的控制权会发生变化并执行对应的预设失控行为（如进入锁定关节站立、趴下、整机急停等）。

3、控制权主动切换的场景：SDK 主动释放控制权、遥控器抢夺控制权、遥控器主动释放。

4、控制权被动切换的场景：低电量、SDK 异常断联、在没有任何SDK连接时遥控器断联、关节电机异常(包括过温、过载、编码器错误等)、STO被按下。

5、SDK连接成功后，默认没有控制权,可以调用**AstrallAuthControl()**接口申请控制权。

6、当有多个SDK申请控制权时，第一个申请控制权的设备会获得控制权，其他设备返回申请失败。

7、当SDK拥有控制权时，此时操作遥控器进行抢夺控制权操作(三档开关由1调至3)，遥控器会获得控制权，SDK失去控制权， 仅失去控制权，不影响获取之前订阅的信息，但是不能订阅新的信息。

8、当遥控器抢夺控制权后，其他设备无法再申请控制权，需要遥控器主动释放控制权（将三档开关拨回1），释放控制权后，其他设备可以申请控制权。

控制权

```
enum AstrallAuth : uint16_t
{
    ASTRALL_AUTH_SDK                = 1U,           // 1U SDK control operation
    ASTRALL_AUTH_JOYSTICK           = 2U            // 2U Remote control operation
};
```

6.10.3 移动接口

函数名	**AstrallMove**
函数原型	uint16_t AstrallMove(float vx, float vy, float vyaw, uint32_t timeout)
功能描述	泰坦移动控制接口
使用方法	请求/应答
参数	vx：前进速度（单位： m/s） vy：平移速度（单位： m/s） vyaw： 旋转速度（单位： rad/s） timeout：超时时间（单位：ms）
返回值	详见 函数调用返回参数表
备注	速度控制 停止时需要发（.0f，.0f，.0f） 正在移动中如果超 50ms 未接收到移动命令将自动停止

参数范围 vx[-1.0, +1.0] vy[-1.0, +1.0] vyaw[-1.0, +1.0]

6.10.4 灯光控制接口

函数名	**AstrallLightControl**
函数原型	uint16_t AstrallLightControl(AstrallLightCmd cmd, uint32_t timeout)
功能描述	控制泰坦头部探照灯
使用方法	请求/应答

参数	cmd：开关，参考 AstrallLightCmd 类型 timeout：超时时间（单位：ms）
返回值	详见 函数调用返回参数表
备注	无

灯光控制

```
enum AstrallLightCmd : uint16_t
{
    ASTRALL_CMD_LIGHT_OPEN          = 1U,           // 1U turn on
    ASTRALL_CMD_LIGHT_CLOSE         = 2U            // 2U turn off
};
```

7、实例代码

```
int main(int argc, char **argv)
{
    int counter = 0;
    uint16_t res = 0;
    AstrallSystemStatus status;
    AstrallPowerStatus pwrSta;
    AstrallImuData imuData;
    AstrallConfig sdkCfg;
    AstrallDeviceInfo devInfo;
    KeyBoard kb;
    kb.registerEventCb(keyboardHandle);

    sdkCfg.heartbeatCb = heartbeatHandle;
    sdkCfg.sdkStatusCb = sdkStatusHandle;

    if(AstrallSdkInit(sdkCfg) != ASTRALL_RES_SUCCESSED)
    {
        printf("SDK initialization failed\n");
        return -1;
    }

    res = AstrallSubscriptionData(ASTRALL_SUB_TOPIC_ID_IMU, ASTRALL_SUB_FREQ_250HZ, imuDataHandle);
    printf("Subscription sensor data:0x%04X\r\n", res);
    res = AstrallSubscriptionData(ASTRALL_SUB_TOPIC_ID_SPORT, ASTRALL_SUB_FREQ_250HZ, sportDataHandle);
    printf("Subscription sport data:0x%04X\r\n", res);
    while (1)
    {
        if (counter++ >= 5)
        {
            counter = 0;
            res = AstrallHeartbeat();
            printf("Heartbeat,res:0x%04X\r\n", res);
            devInfo = AstrallDeviceInfo();
            res = AstrallGetDeviceInfo(devInfo);
            printf("Device info,res: 0x%04X ver:%s sn:%s\r\n", res, devInfo.version.c_str(), devInfo.serialNumber.c_str());
            status = AstrallSystemStatus();
            res = AstrallGetSystemStatus(status);
            printf("System status,res:0x%04X sta:%u err:0x%04X warn:0x%04X\r\n", res, (uint16_t)status.sysStatus, status.errorCode, status.warnCode);
            res = AstrallGetSportStatus();
            printf("Sport status,res:0x%04X\r\n", res);
            pwrSta = AstrallPowerStatus();
            res = AstrallGetPowerStatus(pwrSta);
            printf("Bat status,res:0x%04X soc:%.1f temp:%.1f vol:%.1f chg:%u cc:%u\r\n", res, pwrSta.soc, pwrSta.temp, pwrSta.voltage, pwrSta.charged, pwrSta.cycleCount);
        }
        if (AstrallGetSportStatus() == ASTRALL_SPORT_STATUS_MOVE)
        {
            res = AstrallMove(vx, vy, vyaw);
            printf("Move,res:0x%04X x:%.1f y:%.1f yaw:%.1f\r\n", res, vx, vy, vyaw);
        }
        usleep(20000);
    }
    AstrallSdkDeinit();
    return 0;
}
```

clark Astralldynamics
2025-11-28 17:54:35